

Transport Layer Protocols

-Dr. Sumana Maiti

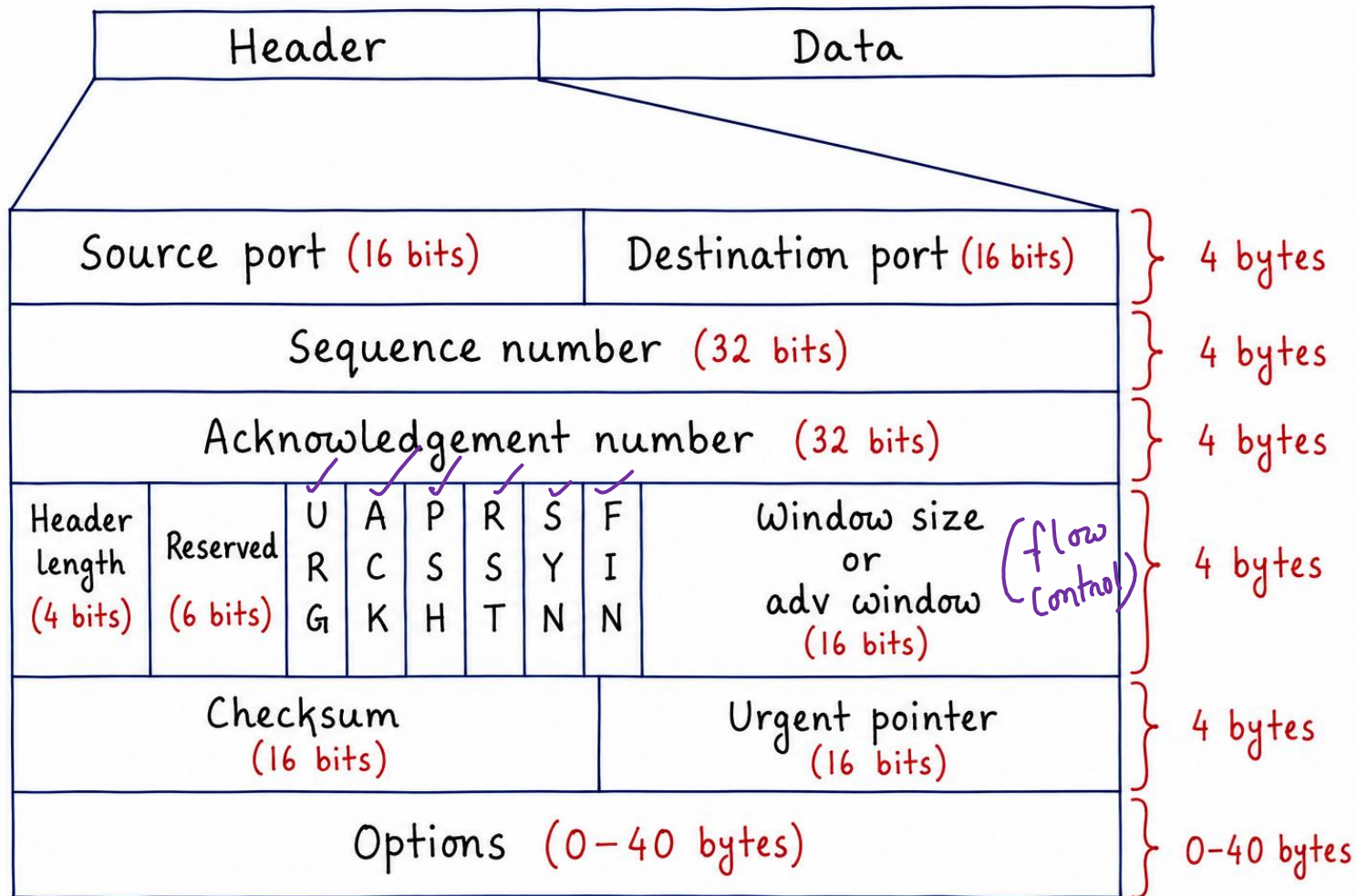
Transport-layer protocol usually has several responsibilities. One is to create a **process-to-process communication**; these protocols use **port numbers** to accomplish this. **Port numbers provide end-to-end addresses at the transport layer and allow multiplexing and demultiplexing at this layer.**

- **TCP (Transmission Control Protocol)**
- **UDP (User Datagram Protocol)**

TCP

- Transmission Control Protocol.
- TCP is a **reliable connection-oriented protocol** that can be used in any application, where reliability is important
- TCP uses **Go-back-N and Selective Repeat for flow control** and **Checksum for error control**.

TCP Header



- Every TCP segment has Header and Data
- Every flag is one bit each

URG, ACK, PSH, RST, SYN, FIN

- Minimum header length is 20 bytes
- Maximum header length is $20+40=60$ bytes

URG: Urgent pointer is valid

ACK: Acknowledgment is valid

PSH: Request for push

RST: Reset the connection

SYN: Synchronize sequence numbers

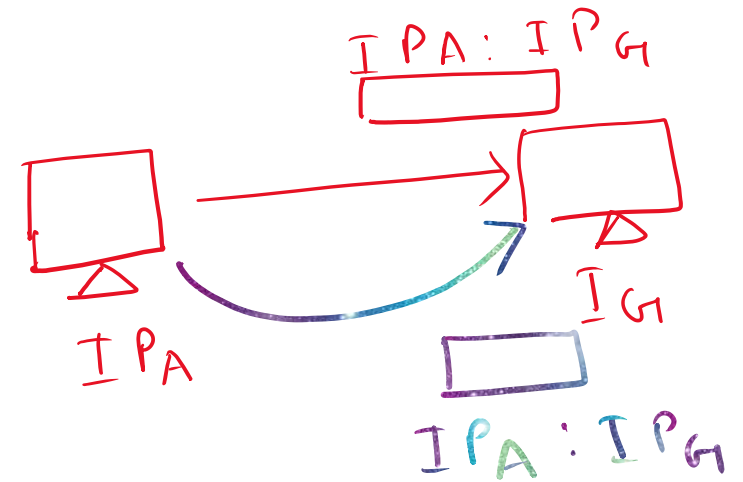
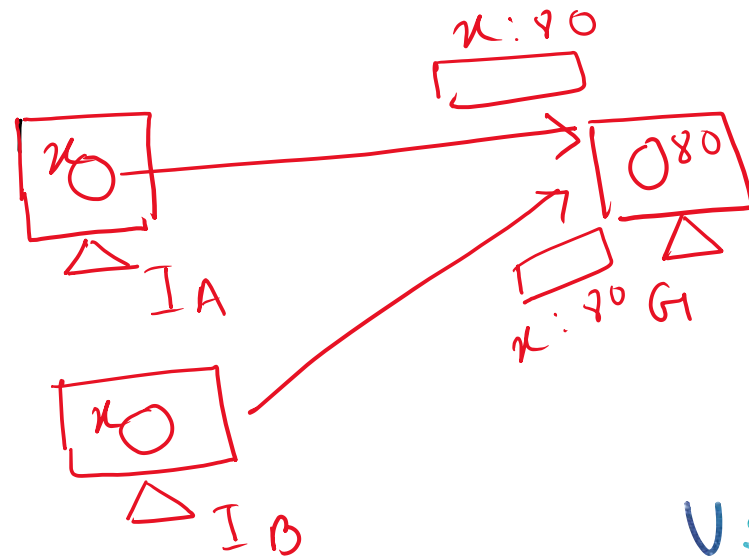
FIN: Terminate the connection

Source Port and Destination Port

- Source port is 16 bits, Destination port is 16 bits.
- $(0 \text{ to } 2^{16}-1) = (0 \text{ to } 65,535)$ port numbers are there.
- Transport layer is end to end connection. Hence, port number is important.
- To identify the process, we need port number.
- Port numbers help in multiplexing and demultiplexing.
- Some port numbers are well known (Range : 0 to 1023)
- HTTP: 80, Telnet: 23, FTP: 21, SMTP: 25
- Some port numbers are reserved (Range 1024 to 49,151).
- Some port numbers are used by general public (Range 49,152 to 65,535)

Socket

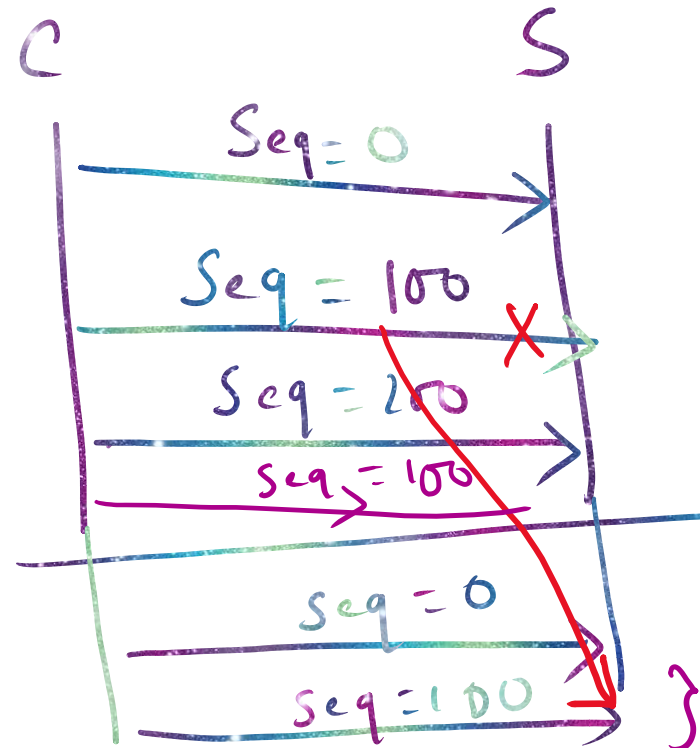
- TCP is connection oriented. Resources are reserved at both side.
- If we use only port number to identify the connection, then if two users use identical port number, then connection cannot be identified.
- If we use only IP address to identify the connection, then if two connections from same host, then connection cannot be identified.
- So, we cannot use only port number or only IP address.
- We use Source port, Destination port, Source IP, Destination IP.
- $IP + Port = Socket$.
- Socket is 48 bits number which is a combination of IP address and Port number and can uniquely identify a connection.



Using only port number
or using only IP, we cannot
distinguish the connection

Sequence number and Acknowledgement number

- TCP is **byte stream protocol**. Every byte sent out is counted.
- From byte number 100 to byte number 150 are present in the segment, then sequence number present in the header is the sequence number of first byte (byte no 100)
- Acknowledgment number is the sequence number of the bytes which is expected next.
- **Acknowledgment number** is “last byte sequence number +1”
- TCP uses **random initial sequence number**



Connection terminated

} It will confuse the server

Q1. A TCP segment contains 1,200 bytes of data and has a sequence number of 10,000. What acknowledgment number should the receiver send if the segment arrives correctly?

Answer:

Acknowledgement number = Sequence number + Data length = 11,200

Q2. An IPv4 packet contains a TCP segment with these values:

IP Total Length = 1,500 bytes, IP Header Length = 20 bytes, TCP Header Length = 20 bytes, TCP Sequence Number = 20,000

Calculate the TCP acknowledgment number.

Answer:

IP Total Length = **1,500 bytes**

IP Header Length = 20 bytes

TCP Header Length = 20 bytes

TCP Sequence Number = **20,000**

TCP data length = $1500 - 20 - 20 = 1460$ bytes

Now calculate the acknowledgment number:

Acknowledgement number = $20,000 + 1,460 = 21,460$

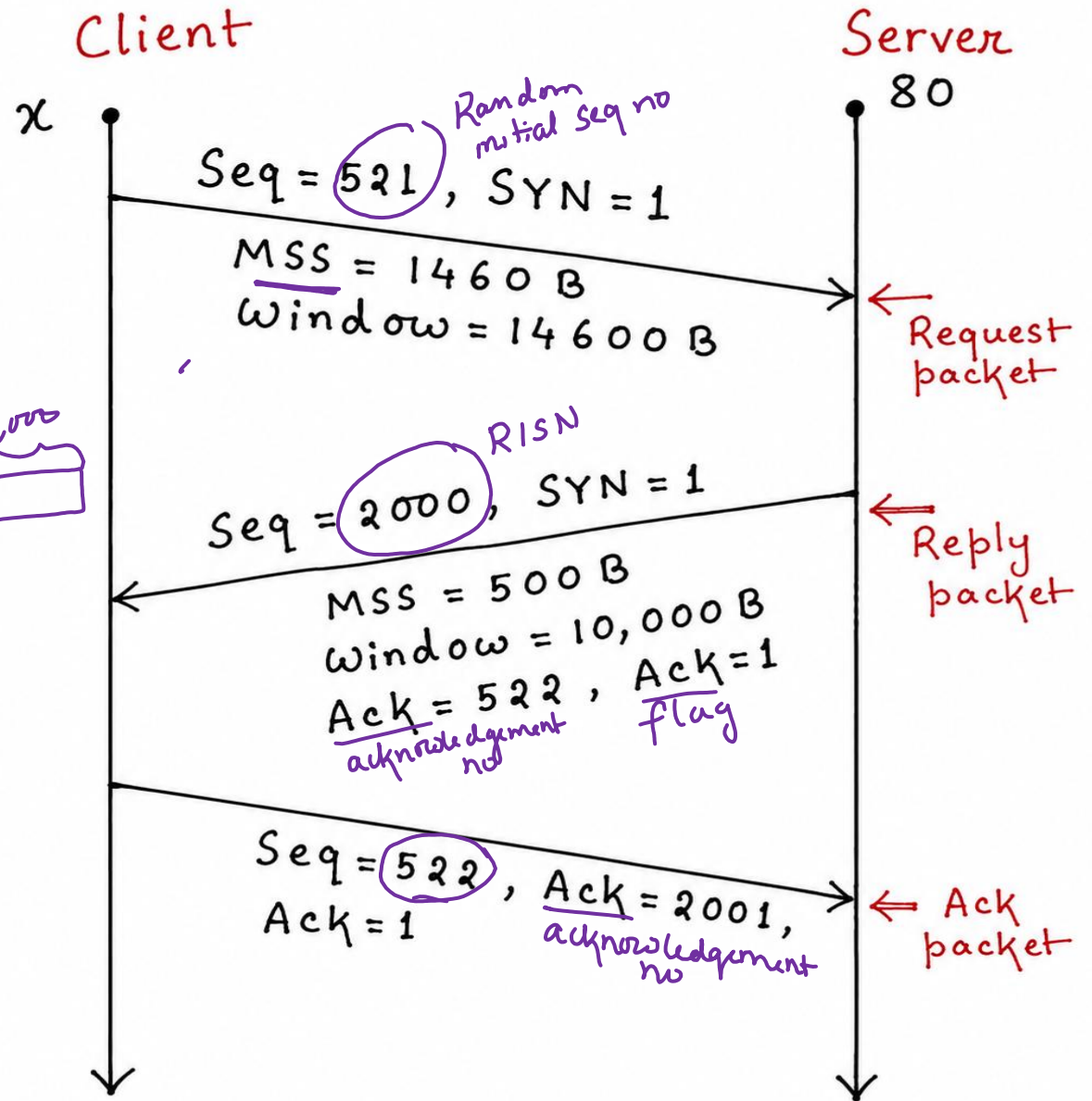
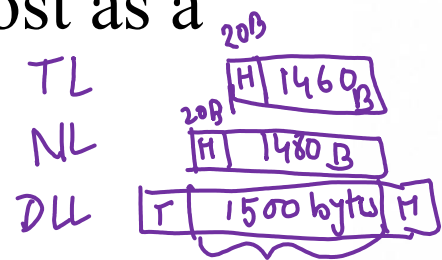
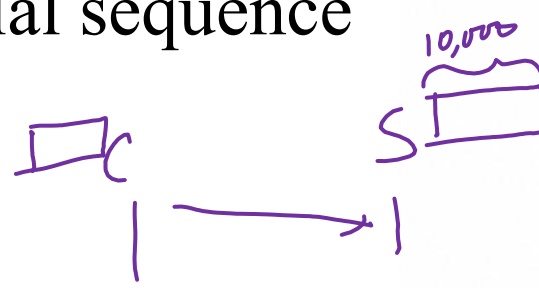
Three Phases

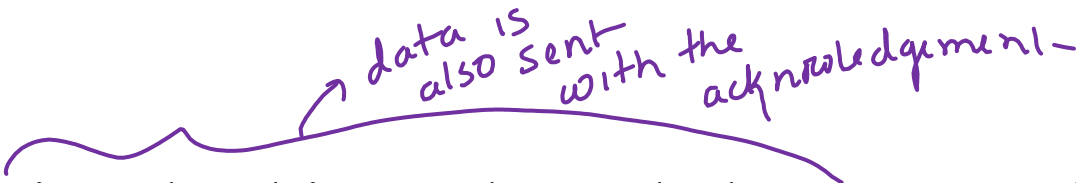
- Connection Establishment
- Data Transfer
- Connection Termination

.

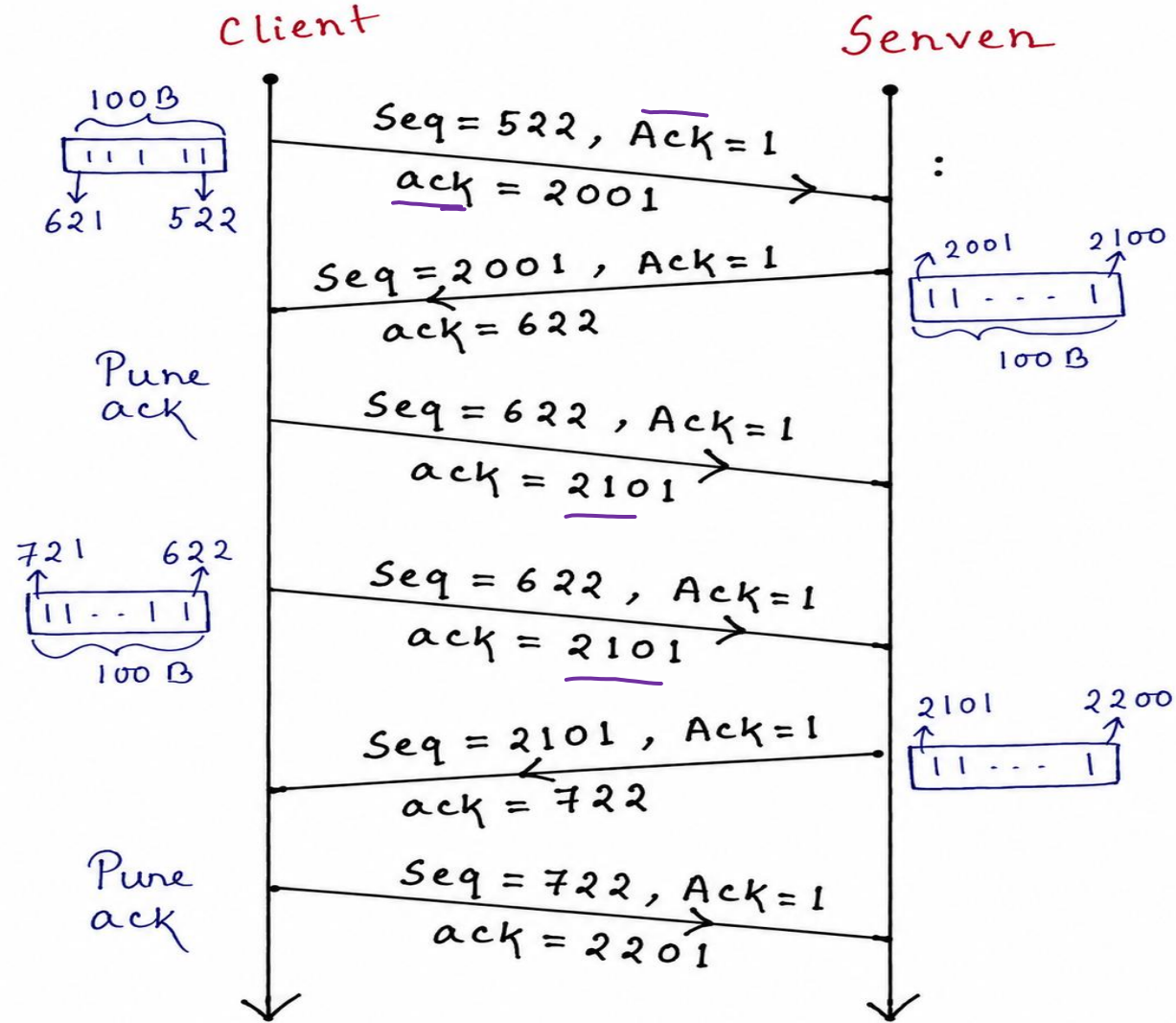
Connection Establishment

- Sequence no ("Seq") contains the initial sequence number.
- SYN flag is for synchronization
- MSS: Maximum segment size
- Client sends Window size, (capacity of the host as a receiver)



- 
- TCP uses both piggybacking acknowledgement and pure acknowledgement
 - Piggybacking: Data packet itself is put in the acknowledgement.
 - SYN=1 : It consumes one sequence number
 - Ack=1: (Pure Acknowledgement) It does not consume any sequence number.
 - FIN=1: It consumes one sequence number.
 - 1 data byte consumes one sequence number.

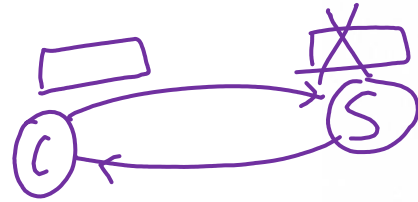
Data Transfer



SYN	ACK	
1	0	First segment or request segment
1	1	Second segment or reply segment
0	1	Ack is present in the header
0	0	Not possible

Connection Termination

FIN=1: Host wants to finish the connection, It does not have any packet to send.

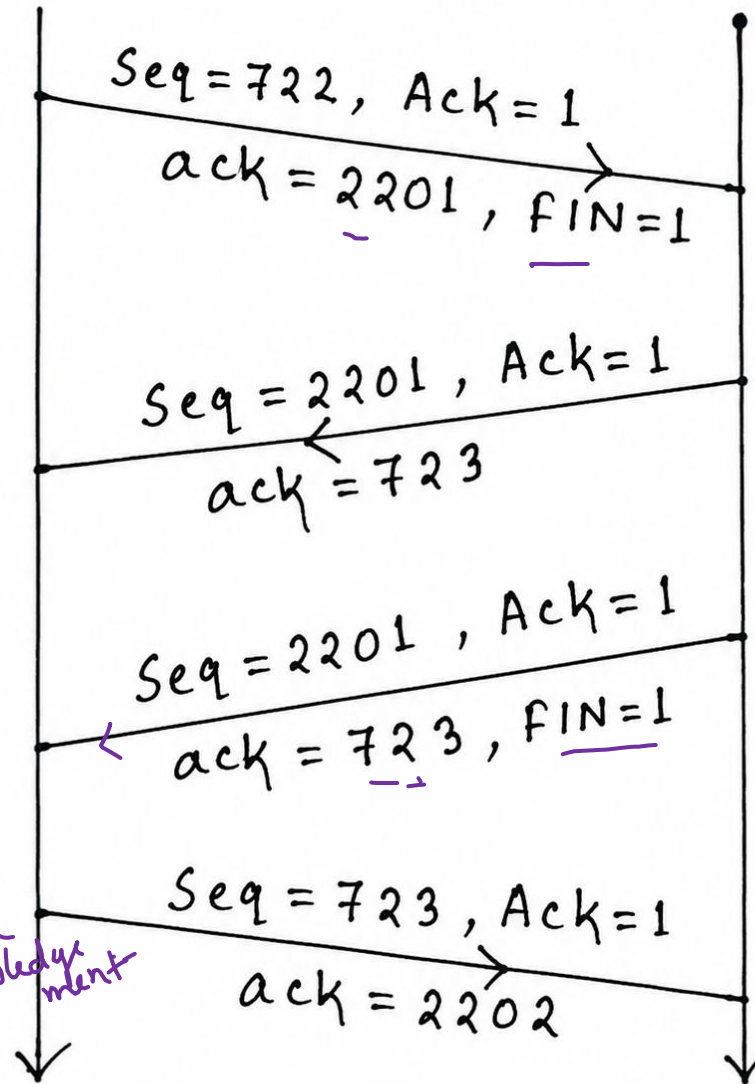


$C \rightarrow S$ X
 $S \rightarrow C$ ✓
 $C \xrightarrow{Ack} S$ ✓
 $S \rightarrow C$ ✓

pure acknowledgment

Client

Server



Pure acknowledgment

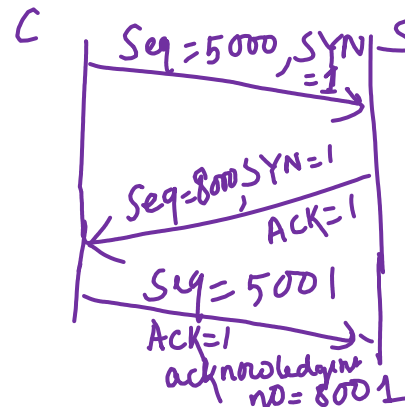
Q3. A TCP sender transmits segments with sequence numbers 1,001, 2,001, and 3,001, each carrying 1,000 bytes. If the second segment is lost but the third arrives, what acknowledgment number will the receiver send?

Answer: It sends a duplicate acknowledgment for 2,001.

Q4. During a TCP three-way handshake:

1. The client sends a **SYN** segment with
Seq=5,000
2. The server replies with a **SYN-ACK** segment with
Seq=8,000.

Find the sequence number and acknowledgment number of the client's final ACK segment.



Answer: Sequence number: 5001, Acknowledgement: 8001

UDP

- ❑ User Datagram Protocol
- ❑ Need of UDP:
 - ❑ *There are some specific application, where TCP is overhead.*
 - ❑ *Broadcasting: If TCP, resources need to be reserved for every connection.*
 - ❑ *Some applications need ~~speed~~ than reliability.*
- ❑ UDP header

(DNS)

Source port (16 bits), Destination port (16 bits), Length (16 bits),
Checksum (16 bits)

Transport layer takes the datagram and give it to user. This is why it is called user datagram protocol.

